



ANTALYA BELEK ÜNİVERSİTESİ

MÜHENDİSLİK MİMARLIK FAKÜLTESİ YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

Utku Yaşar

210007020

LİSANSÜSTÜ BAŞVURU SİSTEMİ

YAZILIM MÜHENDİSLİĞİ LİSANS PROGRAMI TASARIM PROJESİ

Mezuniyet Projesi Değerlendirme Jürisi

Dr. Öğr. Üyesi Kerim Yıldırım

ÖNSÖZ.....	3
1.GİRİŞ VE PROJE TANIMI.....	4
1.1.DÖKÜMANIN AMACI	5
1.2.SDLC TABLOSU	5
1.3.PROJENİN ÖZETİ	6
1.4.MEVcut SORUNLAR VE ÇÖZÜM YAKLAŞIMI	6
2.PROJE KAPSAMI	7
2.1.ADAY MODÜLÜ	7
2.2.KOMİSYON JÜRİ MODÜLÜ.....	8
2.3.YÖNETİCİ MODÜLÜ	9
2.4.BİLDİRİM VE ENTEGRASYON MODÜLÜ.....	10
3. HEDEF KULLANICILAR VE İHTİYAÇ ANALİZİ	10
3.1.ADAY(ÖĞRENCİ)	10
3.2.KOMİSYON ÜYESİ.....	11
3.3.SİSTEM YÖNETİCİSİ	11
4.FONKSİYONEL GEREKSİNİMLER.....	12
4.1. KİMLİK DOĞRULAMA VE YETKİLENDİRME	12
4.2. BAŞVURU YÖNETİMİ	12
4.3. DEĞERLENDİRME SÜRECİ.....	13
4.4. BİLDİRİM SİSTEMİ	13
5. FONKSİYONEL OLMAYAN GEREKSİNİMLER (KALİTE NİTELİKLERİ).....	13
5.1. KULLANILABİLİRLİK.....	13
5.2. PERFORMANS VE ÖLÇEKLENEBİLİRLİK	14
5.3. GÜVENLİK.....	14
5.4. VERİ BÜTÜNLÜĞÜ	14
6. VERİ GEREKSİNİMLERİ VE VERİ MODELİ TASLAĞI	14
6.1.KULLANICI VERİLERİ.....	15

6.2. ADAY PROFİLİ VERİLERİ.....	15
6.3. BAŞVURU VERİLERİ.....	15
7. İŞ AKIŞLARI VE SÜREÇ TASARIMI.....	16
Sistemin genel çalışma mantığı adım adım aşağıdaki gibi tasarlanmıştır.....	16
7.1. ADAY BAŞVURU SİSTEMİ	16
7.2.DEĞERLENDİRME AKIŞI	16
7.3. SONUÇLANDIRMA AKIŞI	17
7.4. YÖNETİCİ YÖNETİM AKIŞI	17
8.DİYAGRAMLAR	17
8.1.ER DİYAGRAMI	18
8.2.USE CASE DİYAGRAMI	21
9. MİMARİ TASARIMI VE MONOLİTİK YAPI	23
9.1. MODÜLLER VE SORUMLULUKLARI	24
Auth Modülü (Kimlik Doğrulama).....	24
User Modülü (Kullanıcı Yönetimi)	25
Program Modülü	25
Application Modülü (Başvuru Yönetimi)	26
Document Modülü (Belge Yönetimi)	26
Evaluation Modülü (Değerlendirme)	27
Notification Modülü (Bildirim)	27
Admin Modülü (Yönetim)	28
GENEL SİSTEM YAPISI	28
9.1. MİKROSERVİSLER VE SORUMLULUKLARI	29
9.2. FRONTEND UYGULAMASI	33
9.3. SERVİS ŞEMASI.....	33
10. KULLANILAN TEKNOLOJİLER VE ARAÇLAR.....	34
10.1. FRONTEND TEKNOLOJİLERİ.....	35
10.2. BACKEND TEKNOLOJİLERİ.....	35
10.3. VERİ TABANI VE ALTYAPI.....	36

10.4. Diğer Araçlar ve Altyapı.....	36
11.GELİŞTİRME SÜRECİ.....	37
11.1 Varsayılan Sunucu Adresi	37
11.2. Projenin Kısa Tanımı.....	37
11.3. Teknoloji Özet Tablosu	37
11.4. Ortam Değişkenleri Tablosu.....	38
11.5. Genel Uygulama Ayarları (main.ts)	38
11.6. Veri Modeli İlişki Özeti	39
11.7. Modül Bazında Açıklama ve API Tablosu	39
7.1. AppModule + AppController	39
7.2. AuthModule — Controller: auth	40
7.3. RolesModule — Controller: roles	40
7.4. UsersModule — Controller: users	41
7.5. InstituteModule — Controller: institutes (sınıf düzeyinde JWT)	41
7.6. ProgramsModule — Controller: programs (sınıf düzeyinde JWT)	41
7.7. ApplicationsModule — Controller: applications (sınıf düzeyinde JWT).....	42
7.8. DocumentsModule — Controller: upload	42
7.9. DesicionModule — Controller: desicions	43
11.8. Modül Dışı Genel Yapılar	43
11.9. Modüller Arası İlişkiler ve Bağımlılıklar	43
11.10. Her Modülün Sorumluluğu.....	44
11.11. Tipik Süreç Akışı.....	44
KAYNAKÇA	46

ÖNSÖZ

Günümüzde yükseköğretim kurumlarındaki dijital dönüşüm, idari süreçlerin hızlanması ve adaylara şeffaf bir başvuru deneyimi sunulması açısından kritik bir öneme sahiptir. Bu çalışma, üniversitelerin lisansüstü programlarına yapılan başvuruların manuel yöntemlerden (e-posta veya fiziksel evrak) kurtarılarak tamamen dijital ortama taşınmasını hedefleyen kapsamlı bir web platformu tasarımıdır.

Proje kapsamında, adayların kullanıcı dostu bir arayüzle belgelerini yükleyebilecekleri, akademik jüri üyelerinin başvuruları adil bir şekilde puanlayabileceği ve yöneticilerin tüm süreci merkezi bir panelden denetleyebileceği modüler bir yapı kurgulanmıştır. Modern yazılım geliştirme standartlarına uygun olarak tasarlanan bu sistem; mikroservis mimarisi, yüksek güvenlikli kimlik doğrulama yöntemleri (JWT, 2FA) ve ölçeklenebilir bir veritabanı altyapısı (PostgreSQL) üzerine inşa edilmiştir.

Bu doküman, gereksinim analizinden sistem mimarisine, veritabanı diyagramlarından kullanılacak teknoloji yığınınına kadar projenin tüm tasarım aşamalarını detaylandırmaktadır.

Utku Yaşar

1.GİRİŞ VE PROJE TANIMI

1.1.DÖKÜMANIN AMACI

Bu doküman,Gereksinim Analizi fazının çıktılarını, projenin teknik ve idari kapsamını, kullanıcı ihtiyaçlarını ve sistem gereksinimlerini detaylı bir şekilde tanımlamak amacıyla hazırlanmıştır. Doküman, yazılım geliştirme yaşam döngüsünün (SDLC) temelini oluşturacak ve sonraki tasarım aşamalarına rehberlik edecektir.

1.2.SDLC TABLOSU

Yazılım geliştirme yaşam döngüsünün (SDLC) temel aşamaları doğrultusunda proje planı oluşturulmuştur [1], [2].

Tarih	SDLC Aşaması	Yapılacak İşler
05.11.2025	Gereksinim Analizi	Proje kapsamını belirleme, kullanıcı ihtiyaçları, veri gereksinimleri ¹ .
12.11.2025	Gereksinim Analizi	Fonksiyonel ve fonksiyonel olmayan gereksinimler, iş akışlarını tasarlama ² .
19.11.2025	Vize Haftası	(Akademik Takvim Arası) ³ .
26.11.2025	Tasarım	Veri tabanı tasarımı (PostgreSQL), ER diyagramları ⁴ .
03.12.2025	Tasarım	Backend mimarisi (NestJS), modüller ve servis yapısı tasarımı ⁵ .

17.12.2025	Tasarım	Frontend tasarımı (React), sayfa akışları ve UI mockup ⁶ .
24.12.2025	Geliştirme	NestJS temel modüllerini oluşturma, auth modülü ⁷ .
11.02.2026	Geliştirme	Kullanıcı başvuru modülü ve form validation ⁸ .
18.02.2026	Geliştirme	Yönetici paneli ve onay süreci backend geliştirmeleri ⁹ .
25.02.2026	Geliştirme	React frontend: Giriş, başvuru formu ve listeleme sayfaları ¹⁰ .
04.03.2026	Geliştirme	React frontend: Yönetici paneli ve onay süreçleri UI ¹¹ .
11.03.2026	Geliştirme	PostgreSQL veri tabanı entegrasyonu, NestJS servislerle bağlama ¹² .

1.3.PROJENİN ÖZETİ

Lisansüstü Başvuru Sistemi, üniversiteler bünyesindeki enstitülerin yüksek lisans ve doktora programlarına yapılacak başvuruların tamamen dijital ortamda alınmasını, yönetilmesini ve sonuçlandırılmasını sağlayan web tabanlı bir platformdur[2]. Geleneksel yöntemlerin yerini alacak bu sistem, adayların internet üzerinden başvuru yapmasına, belgelerini dijital olarak yüklemesine ve başvuru durumlarını şeffaf bir şekilde takip etmesine olanak tanır .

1.4.MEVCUT SORUNLAR VE ÇÖZÜM YAKLAŞIMI

Mevcut durumda birçok üniversite, başvuruları e-posta yoluyla veya elden teslim edilen fiziksel belgelerle kabul etmektedir. Bu manuel süreç aşağıdaki kritik sorunlara yol açmaktadır:

Veri Dağınıklığı: Başvuru süreçleri farklı birimlerde koordine edilmekte, bu da veri tutarsızlığına neden olmaktadır.

Zaman Kaybı ve Hata Riski: Adaylar için karmaşık olan bu süreç, komisyon üyeleri için de başvuruların takibini zorlaştırmakta, belge eksikliklerine ve değerlendirme hatalarına zemin hazırlamaktadır.

Geç Bildirimler: Manuel değerlendirme yükü nedeniyle sonuçların açıklanması gecikmektedir.

Projenin Çözümü: Önerilen sistem, tüm bu süreçleri tek bir merkezi platformda toplayarak adaylar ve kurumlar için hızlı, güvenilir ve şeffaf bir yapı sunmayı hedefler. [2]

2.PROJE KAPSAMI

Proje kapsamı, sistemin sınırlarını ve içerdiği temel modülleri belirler. Bu sistemin kapsamı, 1. ve 2. hafta analiz çalışmaları sonucunda aşağıdaki ana modüllerden oluşacak şekilde netleştirilmiştir.

2.1.ADAY MODÜLÜ

Adayların sisteme kayıt olmasını, başvuru süreçlerini yönetmesini ve kişisel verilerini güvenli bir şekilde işlemesini sağlayan temel modüldür. Bu modül, kullanıcı odaklı bir tasarım benimseyerek, adayların başvuru sürecini hızlı ve sorunsuz tamamlamasını amaçlar. Modül, GDPR ve KVKK gibi veri koruma standartlarına uyumlu olacak şekilde geliştirilecektir. [14], [15]

- Kayıt ve Giriş İşlemleri: Adayların güvenli bir şekilde hesap oluşturması ve sisteme giriş yapması. Bu kapsamda, e-posta doğrulama, iki faktörlü kimlik doğrulama (2FA)

seenekleri eklenecektir. Ayrıca, mevcut üniversite hesaplarıyla entegrasyon sağlanarak tekrar kayıt ihtiyacı minimize edilecektir.

- **Başvuru Formu Yönetimi:** Kişisel bilgilerin (ad, soyad, iletişim bilgileri, eğitim geçmişı) girilmesi, program seçimlerinin yapılması. Form, dinamik alanlara sahip olacak; örneğin, seçilen programa göre ek sorular (araştırma alanı tercihleri, motivasyon mektubu) otomatik olarak görüntülenecektir. Hata kontrolleri (örneğin, zorunlu alanlar, format doğrulamaları) ve otomatik kaydetme özelliği ile kullanıcı dostu hale getirilecektir.
- **Belge Yükleme Sistemi:** Transkript, ALES belgesi, dil belgesi (örneğin, YDS, TOEFL), özgeçmiş, referans mektupları gibi zorunlu ve opsiyonel evrakların dijital olarak sisteme yüklenmesi. Sistem, dosya formatlarını (PDF, JPEG, PNG) kısıtlayacak, dosya boyutlarını sınırlandıracak (maks. 5MB/dosya) ve OCR (Optik Karakter Tanıma) entegrasyonu ile belgelerin otomatik olarak taranmasını sağlayacaktır. Yüklenen belgelerin ön izlemesi ve silme/düzenleme seçenekleri eklenecektir.
- **Durum Takibi:** Başvurunun hangi aşamada olduğunu (Başvuru Alındı, Değerlendiriliyor, Onaylandı, Reddedildi, Beklemede) gerçek zamanlı görüntülenmesi. Adaylar, bir dashboard üzerinden ilerlemeyi grafiksel olarak (örneğin, ilerleme çubuğu) takip edebilecek ve geçmiş başvurularını arşivleyebilecekler. Ayrıca, başvuru revizyonu için sınırlı süreli düzenleme hakkı verilecektir.

2.2.KOMİSYON JÜRİ MODÜLÜ

Başvuruları değerlendirecek akademik personelin (jüri üyelerinin) kullanacağı arayüzdür. Bu modül, değerlendirme sürecini standartlaştırarak adil ve verimli bir inceleme sağlamayı hedefler. Erişim yetkileri rol bazlı olacak ve her üye yalnızca kendisine atanan başvuruları görebilecektir.

- **Başvuru Listeleme:** İlgili programa yapılan başvuruların filtrelenmesi (örneğin, puan aralığına, başvuru tarihine, aday adına göre) ve listelenmesi. Liste, tablo formatında olacak; sütunlar özelleştirilebilir (sıralama, arama) ve dışa aktarma seçenekleri (CSV, PDF) eklenecektir. Toplu işlem özellikleri (örneğin, birden fazla başvuruyu aynı anda atama) dahil edilecektir.

- **Belge İnceleme:** Adayların yüklediği belgelerin sistem üzerinden görüntülenmesi ve ön izlemesi. Belgeler, tarayıcı tabanlı bir viewer ile indirilmeden görüntülenebilecek; zoom, döndürme ve not ekleme gibi özellikler eklenecektir.
- **Değerlendirme ve Puanlama:** Başvuruların kriterlere (örneğin, not ortalaması %40, ALES puanı %30, mülakat %30) göre puanlanması, öneri veya ret kararlarının sisteme girilmesi. Puanlama formu otomatik hesaplamalar yapacak (örneğin, ağırlıklı ortalama), yorum alanları eklenecek ve kararlar için onay zinciri (örneğin, birden fazla üyenin onayı) tanımlanacaktır. Raporlama araçları ile istatistiksel analiz (örneğin, ortalama puanlar) üretilecektir.

2.3.YÖNETİCİ MODÜLÜ

Sistemin genel ayarlarının, kullanıcı yönetiminin ve süreç onaylarının yapıldığı merkezi kontrol panelidir. Bu modül, sistemin bakımını ve ölçeklenebilirliğini sağlamak üzere tasarlanmıştır. Ek olarak, yönetici rollerinin yönetimi için bir hiyerarşi tanımlanmıştır: Sistemde bir "Süper Yönetici" rolü tanımlanacak olup, bu rol yalnızca geliştirici ekip tarafından atanabilecek ve diğer yöneticilerin (admin) yetkilerini düzenleyebilecek şekilde yapılandırılacaktır. Bu, yetki çatışmalarını önlemek ve güvenlik katmanı eklemek amacıyla yapılmıştır.

Program ve Kontenjan Tanımlama: Açılacak lisansüstü programların (yüksek lisans, doktora, entegre programlar), kontenjanların (yerli/yabancı ayrımı), başvuru takvimlerinin ve ücret bilgilerinin sisteme girilmesi. Dinamik formlarla yeni program ekleme, kontenjan güncelleme ve silme işlemleri desteklenecektir. Ayrıca, program kriterleri (minimum puanlar, ön koşullar) tanımlanabilecektir.

Süreç İzleme: Tüm başvuru ve değerlendirme süreçlerinin kuş bakışı izlenmesi. Dashboard, gerçek zamanlı metrikler (başvuru sayısı, değerlendirme oranı, ortalama işlem süresi) gösterecek; grafikler (çubuk, pasta) ve uyarı sistemleri (örneğin, geciken değerlendirmeler için bildirim) eklenecektir. Log kaydı ile tüm eylemler takip edilecektir.

Sonuç Onaylama: Komisyon kararlarının nihai onaya sunulması ve yayınlanması. Onay süreci, çok seviyeli (örneğin, yönetici ve dekan onayı) olacak; sonuçlar toplu olarak yayınlanabilecek ve yedek listeler yönetilebilecektir. Rapor üretimi (örneğin, kabul/red istatistikleri) ve arşivleme özellikleri dahil edilecektir.

Kullanıcı ve Yetki Yönetimi: Sistem yöneticileri (admin), diğer kullanıcı rollerini (aday, komisyon) ekleyip düzenleyebilecek; ancak diğer adminlerin yetkilerini yalnızca Süper Yönetici rolü düzenleyebilecek.

2.4.BİLDİRİM VE ENTEGRASYON MODÜLÜ

Otomatik Eposta Bildirimleri: Başvuru alındığında veya sonuçlandığında adaylara sistem tarafından otomatik e-posta gönderilmesi.

Veritabanı Entegrasyonu: Tüm verilerin PostgreSQL üzerinde tutarlı bir şekilde saklanması.

3. HEDEF KULLANICILAR VE İHTİYAÇ ANALİZİ

Sistemin başarısı, hedef kullanıcıların ihtiyaçlarının tam olarak karşılanmasına bağlıdır. Analiz sürecinde üç temel kullanıcı rolü belirlenmiştir.

3.1.ADAY(ÖĞRENCİ)

- Profil: Yüksek lisans veya doktora programına başvurmak isteyen mezun veya son sınıf öğrencileri.

- **Temel Sorumluluklar:** Kişisel bilgilerini girer, belgelerini yükler ve başvuru durumunu takip eder.
- **İhtiyaçlar:**
 - Fiziksel evrak taşımadan, internet olan her yerden başvuru yapabilmek.
 - Yüklediği belgelerin sisteme ulaştığından emin olmak.
 - Başvuru sonucunu e-posta veya panel üzerinden anlık olarak öğrenebilmek.
 - Kullanıcı dostu, karmaşık olmayan bir arayüz (UI) kullanmak.

3.2.KOMİSYON ÜYESİ

- **Profil:** Başvuruları değerlendirmekle görevli öğretim üyeleri.
- **Temel Sorumluluklar:** Başvuruları inceler, puanlama yapar, öneri veya ret kararı verir.
- **İhtiyaçlar:**
 - Kendisine atanan başvuruları tek bir ekranda toplu halde görebilmek.
 - Belgeleri indirmeden tarayıcı üzerinde ön izleyebilmek.
 - Puanlama yaparken sistemin otomatik hesaplama desteği sunması.
 - Kağıt israfı ve dosya karmaşasından kurtulmak.

3.3.SİSTEM YÖNETİCİSİ

- **Profil:** Başvuru takvimini ve programları yöneten idari personel.
- **Temel Sorumluluklar:** Program tanımlar, kontenjan belirler, süreçleri izler ve sonuçları onaylar.
- **İhtiyaçlar:**
 - Sisteme yeni program ve kontenjan ekleyebilmek.

- o Kullanıcı yetkilendirmelerini yönetmek.
- o Süreç sonunda raporlar alabilmek ve sonuç listelerini yayınlayabilmek.

4.FONKSİYONEL GEREKSİNİMLER

Sistemin teknik olarak yerine getirmesi gereken işlevler aşağıda detaylandırılmıştır. Bu gereksinimler 3. haftadan itibaren başlayacak Tasarım ve Geliştirme aşamalarına girdi oluşturacaktır.

4.1. KİMLİK DOĞRULAMA VE YETKİLENDİRME

- **FR-01:** Sistem, adayların e-posta ve şifre ile kayıt olmasına izin vermelidir.
- **FR-02:** Kullanıcı girişleri **JWT (JSON Web Token)** kullanılarak güvenli bir şekilde doğrulanmalıdır.[6], [7]
- **FR-03:** Sistem; Aday, Komisyon Üyesi ve Yönetici olmak üzere üç farklı rolü desteklemeli ve sayfa erişimlerini bu rollere göre kısıtlamalıdır.
- **FR-03a:** Sistemde bir "Süper Yönetici" alt rolü tanımlanmalı; bu rol, diğer yöneticilerin (admin) eklenmesi, silinmesi veya yetki düzenlemesini sağlayacak şekilde yapılandırılmalıdır. Süper Yönetici ataması yalnızca sistem geliştiricileri tarafından yapılabilmelidir.

4.2. BAŞVURU YÖNETİMİ

- **FR-04:** Adaylar, kişisel bilgilerini (Ad, Soyad, T.C. Kimlik No, Adres vb.) form alanlarına girebilmelidir.

- **FR-05:** Sistem, form validasyonları (doğrulama) yaparak eksik veya hatalı veri girişini engellemelidir (Örn: T.C. No kontrolü).
- **FR-06:** Adaylar, PDF veya JPEG formatındaki belgelerini sisteme yükleyebilmelidir.
- **FR-07:** Adaylar, başvuru süresi bitene kadar başvurularını güncelleyebilmelidir.

4.3. DEĞERLENDİRME SÜRECİ

- **FR-08:** Komisyon üyeleri, kendilerine atanan başvuruları listeleyebilmelidir.
- **FR-09:** Komisyon üyeleri, her başvuru için "Kabul", "Red" veya "Mülakata Çağır" şeklinde durum güncellemesi yapabilmelidir.
- **FR-10:** Yönetici, komisyon tarafından onaylanan listeyi inceleyip nihai onay verebilmelidir.

4.4. BİLDİRİM SİSTEMİ

- **FR-11:** Sistem, başvuru tamamlandığında adaya "Başvurunuz Alındı" e-postası göndermelidir.
- **FR-12:** Sonuçlar açıklandığında adaylara durum bildiren otomatik e-posta gönderilmelidir.
- **FR-13:** E-posta kuyruk yönetimi için **RabbitMQ** teknolojisi kullanılmalıdır.

5. FONKSİYONEL OLMAYAN GEREKSİNİMLER (KALİTE NİTELİKLERİ)

Sistemin *nasıl* çalışacağını belirleyen kalite kriterleridir.

5.1. KULLANILABİLİRLİK

- Arayüzler modern ve kullanıcı dostu olmalıdır.

- Frontend geliřtirmesinde **React** kütüphanesi ve **TailwindCSS** framework'ü kullanılarak responsive (mobil uyumlu) bir tasarım sağlanmalıdır.

5.2. PERFORMANS VE ÖLÇEKLENEBİLİRLİK

- Sistem, aynı anda yüzlerce adayın başvuru yapabileceđi yoğun dönemlerde (başvuru haftaları) performans kaybı yaşamamalıdır.
- Backend mimarisi **NestJS** kullanılarak modüler ve ölçeklenebilir bir yapıda tasarlanmalıdır.

5.3. GÜVENLİK

- Kullanıcı şifreleri veritabanında hash'lenmiş (şifrelenmiş) olarak saklanmalıdır.
- Veri iletimi SSL sertifikası ile şifrelenmelidir.
- Kimlik doğrulama süreçleri JWT standardına uygun olmalıdır.[6], [7]

5.4. VERİ BÜTÜNLÜĐÜ

- Veritabanı olarak **PostgreSQL** kullanılmalı, veriler ilişkisel model (Relational Model) kurallarına göre tutarlı bir şekilde saklanmalıdır.
- Hatalı veri girişlerini önlemek için hem frontend hem de backend tarafında veri doğrulama (validation) yapılmalıdır.

6. VERİ GEREKSİNİMLERİ VE VERİ MODELİ TASLAĐI

Proje kapsamında toplanacak ve işlenecek verilerin yapısı, 4. haftada yapılacak veritabanı tasarımı öncesinde belirlenmiştir. Aşağıdaki ana veri varlıkları (entities) sistemde bulunmalıdır:

6.1.KULLANICI VERİLERİ

- **User ID:** Benzersiz kimlik numarası.
- **Email:** Giriş ve iletişim için.
- **Role:** Kullanıcı rolü (Aday, Komisyon, Admin).

6.2. ADAY PROFİLİ VERİLERİ

- **TC Kimlik No:** Kimlik doğrulama için.
- **Ad Soyad:** Adayın tam adı.
- **Telefon:** İletişim numarası.
- **Mezuniyet Bilgileri:** Mezun olduğu üniversite, ortalama (GANO).

6.3. BAŞVURU VERİLERİ

- **Basvuru ID:** Benzersiz başvuru numarası.
- **Program ID:** Başvurulan program (Yüksek Lisans/Doktora).
- **Basvuru Tarihi:** Başvurunun yapıldığı zaman damgası.
- **Durum:** (Beklemede, Onaylandı, Reddedildi).
- **Belge Yollari:** Yüklenen dosyaların sunucudaki yolları.

7. İŞ AKIŞLARI VE SÜREÇ TASARIMI

Sistemin genel çalışma mantığı adım adım aşağıdaki gibi tasarlanmıştır.

7.1. ADAY BAŞVURU SİSTEMİ

1. **Kayıt:** Aday sisteme girer, e-posta ve şifre belirleyerek kayıt olur. Sistem, e-posta doğrulama linki gönderir.
2. **Doğrulama:** Aday, e-posta doğrulamasını tamamlar ve sisteme giriş yapar. Hata durumunda (örneğin, geçersiz e-posta), hata mesajı görüntülenir ve yeniden deneme hakkı verilir.
3. **Form Doldurma:** Kişisel ve eğitim bilgilerini forma girer. Sistem, otomatik kaydetme ile veri kaybını önler ve dinamik validasyon uygular (örneğin, T.C. No format kontrolü).
4. **Belge Yükleme:** İstenen belgeleri (Transkript, ALES vb.) sisteme yükler. Sistem, dosya formatı ve boyut kontrolü yapar; başarılı yükleme sonrası ön izleme sunar.
5. **Düzenleme:** Aday, başvuru takviminde tanımlanan düzenleme kapanış tarihine kadar formu ve belgeleri düzenleyebilir. Tarih geçerse, sistem otomatik kilitleme uygular ve yöneticiye bildirim gönderir (eğer istisna talep edilirse).
6. **Onay:** Başvuruyu kontrol eder ve "Başvuruyu Tamamla" butonuna basar. Sistem veritabanına kaydeder, otomatik e-posta bildirimi gönderir ve durum "Başvuru Alındı" olarak güncellenir.

7.2. DEĞERLENDİRME AKIŞI

1. **Atama:** Yönetici veya sistem, başvuruyu ilgili komisyon üyelerinin ekranına düşürür. Otomatik atama algoritması (örneğin, yük dengeleme) kullanılabilir.
2. **İnceleme:** Komisyon üyesi sisteme girer, başvuruyu görüntüler ve belgeleri inceler. Sistem, belge viewer ile zoom/not ekleme sağlar; hata durumunda (örneğin, belge açılmama), yöneticiye uyarı gönderir.

3. **Puanlama:** Kriterlere göre puan girer; sistem otomatik ağırlıklı ortalama hesaplar ve yorum alanı sunar.
4. **Karar:** Komisyon üyesi notunu verir ve kararını (Kabul/Red/Mülakat) sisteme işler. Karar, onay zinciri ile birden fazla üye tarafından doğrulanabilir.
5. **Güncelleme:** Değerlendirme tamamlandığında, durum güncellenir ve yöneticiye bildirim gider.

7.3. SONUÇLANDIRMA AKIŞI

1. **Nihai Onay:** Yönetici, komisyon kararlarını kontrol eder ve onaylar. Sistem, toplu onay özelliği sunar; ret durumunda gerekçe zorunlu tutulur.
2. **Yayınlama:** Sonuçlar sistem üzerinde yayınlanır. Yedek listeler otomatik oluşturulur ve web sitesinde erişilebilir hale getirilir.
3. **Bildirim:** Sistem, sonucu adaya otomatik e-posta/SMS ile bildirir. Bildirim, kişiselleştirilmiş şablonlar kullanır ve gecikme durumunda kuyruk yönetimi (RabbitMQ) ile tekrarlanır.

7.4. YÖNETİCİ YÖNETİM AKIŞI

1. **Süper Yönetici Girişi:** Süper Yönetici rolüyle sisteme giriş yapılır.
2. **Admin Listeleme:** Mevcut admin kullanıcılarını listeler; filtreleme ve arama seçenekleri ile.
3. **Düzenleme/Ekleme:** Yeni admin ekler veya mevcut olanın yetkilerini (örneğin, program yönetimi erişimi) düzenler. Değişiklikler loglanır.

8.DİYAGRAMLAR

Lisansüstü Başvuru Sistemi projesinin tasarım aşamasında, sistemin veri yapısını ve kullanıcı etkileşimlerini görselleştirmek amacıyla iki temel diyagram kullanılmıştır: ER (Entity-Relationship) Diyagramı ve Use Case Diyagramı. Bu diyagramlar, projenin veritabanı mimarisini

ve işlevsel akışlarını netleştirerek, geliştirme sürecine rehberlik eder. Aşağıda, her diyagramın amacı, bileşenleri ve sistemdeki rolü detaylı olarak açıklanmıştır. Bu yazı, projenin 3.12.2025 tarihli dokümanından yola çıkılarak hazırlanmış olup, diyagramların metin tabanlı tariflerini temel alır.

8.1.ER DİYAGRAMI

ER Diyagramı, sistemin veritabanı yapısını temsil eder. PostgreSQL tabanlı ilişkisel modelde, veri varlıkları (entities), alanları (attributes) ve ilişkileri (relationships) gösterir. Bu diyagram, verilerin tutarlı ve etkili bir şekilde saklanması sağlar, veri bütünlüğünü korur ve sorgulama işlemlerini optimize eder. Proje kapsamında 11 ana varlık tanımlanmıştır. Diyagram, kullanıcıların, başvuruların ve değerlendirmelerin nasıl bağlantılı olduğunu görselleştirir. [5]

Ana Varlıklar ve İlişkiler:

- **Users (Kullanıcılar):** Sistemdeki tüm hesapların temel tablosu. Alanlar: user_id (primary key), email, password_hash, role (örn. admin, user, commission_member), created_at, updated_at. İlişkiler: 1-1 ile candidate_profiles; 1-N ile applications ve notifications. Bu tablo, kimlik doğrulamanın temelini oluşturur.
- **Candidate_Profiles (Aday Profilleri):** Adayların kişisel bilgilerini tutar. Alanlar: cp_id (primary key), user_id (FK), tc_kimlik_no, ad, soyad, telefon, eposta, mezun_universite, gpa, dogum_tarihi. İlişkiler: 1-1 ile users. Bu tablo, başvuru formlarının veri kaynağıdır.
- **Universities (Üniversiteler):** Üniversite bilgilerini saklar. Alanlar: university_id (primary key), name, country, city, website, created_at, faculties. İlişkiler: 1-N ile departments.
- **Departments (Bölümler):** Bölüm detaylarını içerir. Alanlar: department_id (primary key), university_id (FK), name, head_of_department, contact_email, created_at. İlişkiler: N-1 ile universities; 1-N ile programs.
- **Programs (Programlar):** Lisansüstü programları tanımlar. Alanlar: program_id (primary key), department_id (FK), program_ad, kontenjan, basvuru_baslangic_tarihi, basvuru_bitis_tarihi, degerlendirme_bas_tarihi, detay, son_guncelleme, program_turu. İlişkiler: N-1 ile departments; 1-N ile applications.

- **Applications (Başvurular):** Her başvuruyu kaydeder. Alanlar: application_id (primary key), user_id (FK), program_id (FK), basvuru_tarihi, duzenleme_tarihi, durum, tezavm_id. İlişkiler: N-1 ile users ve programs; 1-N ile documents ve evaluations.
- **Documents (Belgeler):** Yüklenen dosyaları yönetir. Alanlar: document_id (primary key), application_id (FK), file_path, dosya_tipi, yuklenme_tarihi, hash. İlişkiler: N-1 ile applications.
- **Evaluations (Değerlendirmeler):** Komisyon puanlamalarını tutar. Alanlar: evaluation_id (primary key), application_id (FK), komisyon_uye_id, karar, yorum, degerlendirme_tarihi. İlişkiler: N-1 ile applications.
- **Notifications (Bildirimler):** Sistem bildirimlerini saklar. Alanlar: notification_id (primary key), user_id (FK), application_id (opsiyonel FK), mesaj, tip, gonderim_tarihi, okundu_mu. İlişkiler: N-1 ile users ve applications (opsiyonel).
- **Commission_Members (Komisyon Üyeleri):** Değerlendiricilerin bilgilerini içerir. Alanlar: user_id (primary key), komisyon_uye_id, ad, soyad, department_id (FK), email, title. İlişkiler: N-1 ile departments.
- **Admins (Yöneticiler):** Admin rollerini yönetir. Alanlar: admin_id (primary key), user_id, assigned_at, role. İlişkiler: N-1 ile users.

Genel Sistem Akışı ER Diyagramında: Diyagram, kullanıcı kaydından (users) başlayarak profil doldurma (candidate_profiles), başvuru oluşturma (applications), belge yükleme (documents), değerlendirme (evaluations) ve bildirim (notifications) adımlarını zincirleme gösterir. İlişkiler, veri tutarlılığını sağlar; örneğin, bir başvuru silindiğinde ilişkili belgeler otomatik olarak etkilenir.

Bu diyagram, tasarım aşamasında veri modelini optimize etmek için kullanılmış ve geliştirme sürecinde TypeORM ile implemente edilmiştir.



8.2.USE CASE DİYAGRAMI

Use Case Diyagramı, sistemin kullanıcı rollerine göre işlevlerini gösterir. UML standardında hazırlanmış olup, aktörler (kullanıcılar), use case'ler (işlemler) ve ilişkileri (include, extend) vurgular. Bu diyagram, sistemin kullanıcı odaklı tasarımını yansıtır ve gereksinim analizini somutlaştırır. [5]

Ana Aktörler:

- **Aday (Öğrenci):** Başvuru yapan kişi.
- **Komisyon Üyesi (Akademisyen):** Değerlendirme yapan öğretim üyesi.
- **Yönetici:** Program ve süreç yöneten personel.
- **Süper Yönetici:** Admin yetkilerini düzenleyen üst yönetici.
- **Sistem:** Otomatik bildirimleri tetikleyen arka plan aktörü.

Use Case Blokları:

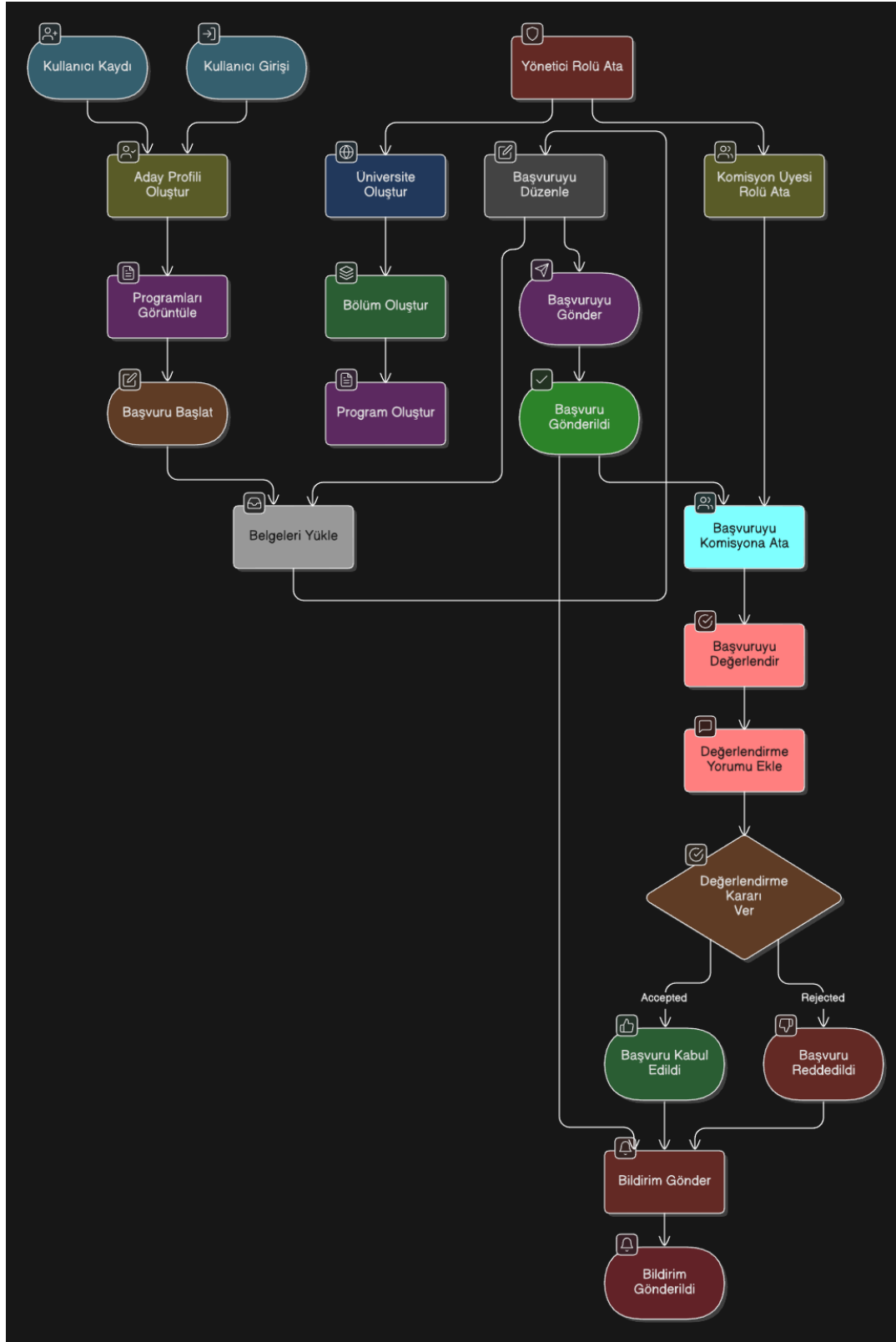
- **Aday İşlemleri:** Kayıt ve Giriş, Başvuru Formu Doldur, Belge Yükleme, Başvuru Güncelleme (extend), Başvuru Durum Takibi.
- **Komisyon İşlemleri:** Başvuru Listeleme, Belge İnceleme (include), Değerlendirme ve Puanlama (include).
- **Yönetici İşlemleri:** Program ve Kontenjan Tanımlama, Süreç İzleme, Kullanıcı ve Yetki Yönetimi, Sonuç Onaylama.
- **Süper Yönetici İşlemleri:** Admin Düzenleme.
- **Sistem İşlemleri:** Otomatik Bildirimler (sonuç onayında, başvuru güncellemesinde vb.).

İlişki Türleri:

- **Include:** Bir işlem her zaman başka bir işlemi içerir (örn. Belge İnceleme → Değerlendirme).
- **Extend:** Bir işlem gerektiğinde genişletilir (örn. Durum Takibi → Sonuç Onaylama).

Genel Akış Use Case Diyagramında: Diyagram, adayların başvuru yapmasından komisyon değerlendirmesine, yönetici onayına ve sistem bildirimlerine kadar tam süreci kapsar. Bu, projenin kullanıcı ihtiyaçlarını karşılamasını sağlar.

Bu diyagramlar, projenin SDLC sürecinde kritik rol oynamış olup, NestJS backend ve React frontend entegrasyonunu kolaylaştırmıştır. Gelecekteki genişletmelerde (örn. mobil entegrasyon), bu diyagramlar temel alınarak güncellenebilir.



9. MİMARİ TASARIMI VE MONOLİTİK YAPI

Lisansüstü Başvuru Sistemi projesinin mimari tasarımı, sistemin geliştirme sürecinde bütünleşik ve merkezi bir yapı ile yönetilmesini sağlamak amacıyla monolitik mimari yaklaşım kullanılarak gerçekleştirilmiştir. Bu yaklaşımda, sistemin tüm bileşenleri tek bir uygulama içerisinde modüler katmanlar halinde yapılandırılmıştır. Böylece geliştirme, test ve dağıtım süreçleri daha sade ve yönetilebilir hale getirilmiştir.

Monolitik yapı içerisinde sistem; kimlik doğrulama, kullanıcı yönetimi, başvuru işlemleri, belge yönetimi ve değerlendirme süreçleri gibi farklı işlevsel modüllerden oluşmaktadır. Bu modüller aynı kod tabanı içerisinde yer almakta olup, birbirleriyle doğrudan fonksiyon çağrıları üzerinden iletişim kurmaktadır. Sistem dışı iletişim REST API üzerinden sağlanmakta ve tüm istekler tek bir backend uygulaması tarafından karşılanmaktadır.

Proje kapsamında NestJS framework'ü kullanılarak katmanlı bir mimari (Controller–Service–Repository) uygulanmıştır. Frontend tarafında React kullanılarak kullanıcı arayüzü geliştirilmiş, veritabanı olarak PostgreSQL tercih edilmiştir. Veri erişim işlemleri TypeORM aracılığıyla gerçekleştirilmiştir.

9.1. MODÜLLER VE SORUMLULUKLARI

Monolitik yapı içerisinde sistem, mantıksal olarak modüllere ayrılmıştır. Her modül belirli bir işlevi yerine getirmekte olup, uygulama içerisinde bağımsız birim gibi organize edilmiştir.

Auth Modülü (Kimlik Doğrulama)

Kullanıcı kimlik doğrulama ve yetkilendirme işlemlerini yönetir.

Teknolojiler: NestJS, TypeORM, PostgreSQL, JWT

Sorumluluklar:

- Kullanıcı kayıt ve giriş işlemleri
- JWT token üretimi ve doğrulama
- Rol bazlı yetkilendirme

- E-posta doğrulama ve şifre sıfırlama

Ana Rotalar:

- POST /auth/register
- POST /auth/login
- POST /auth/verify-email

User Modülü (Kullanıcı Yönetimi)

Kullanıcı profilleri ve rol yönetimini sağlar.

Teknolojiler: NestJS, TypeORM, PostgreSQL

Sorumluluklar:

- Kullanıcı profil yönetimi
- Rol atama ve güncelleme
- Kullanıcı listeleme

Ana Rotalar:

- GET /users/profile
- PUT /users/profile
- GET /users
- PUT /users/:id/role

Program Modülü

Lisansüstü programların yönetimini sağlar.

Teknolojiler: NestJS, TypeORM, PostgreSQL

Sorumluluklar:

- Program tanımlama
- Kontenjan yönetimi
- Başvuru takvimi belirleme

Ana Rotalar:

- GET /programs
- POST /programs
- PUT /programs/:id
- DELETE /programs/:id

Application Modülü (Başvuru Yönetimi)

Başvuru süreçlerinin yönetiminden sorumludur.

Teknolojiler: NestJS, TypeORM, PostgreSQL

Sorumluluklar:

- Başvuru oluşturma ve güncelleme
- Başvuru durumu takibi
- Form validasyonu

Ana Rotalar:

- POST /applications
- GET /applications/my
- GET /applications/:id
- PUT /applications/:id
- GET /applications/assigned

Document Modülü (Belge Yönetimi)

Kullanıcı belgelerinin yüklenmesi ve yönetimini sağlar.

Teknolojiler: NestJS, Multer, PostgreSQL

Sorumluluklar:

- Belge yükleme
- Dosya doğrulama (format, boyut)
- Belge görüntüleme ve silme

Ana Rotalar:

- POST /documents/upload
- GET /documents/:id/view
- DELETE /documents/:id
- GET /applications/:id/documents

Evaluation Modülü (Değerlendirme)

Komisyon değerlendirme süreçlerini yönetir.

Teknolojiler: NestJS, TypeORM, PostgreSQL

Sorumluluklar:

- Başvuruların puanlanması
- Karar oluşturma
- Değerlendirme güncelleme

Ana Rotalar:

- GET /evaluations/my
- POST /evaluations
- PUT /evaluations/:id

Notification Modülü (Bildirim)

Sistem içi bildirim süreçlerini yönetir.

Teknolojiler: NestJS, Nodemailer

Sorumluluklar:

- E-posta bildirimleri gönderme
- Bildirim kayıtlarını tutma

Ana Rotalar:

- GET /notifications

- PUT /notifications/:id/read

Admin Modülü (Yönetim)

Sistem yönetimi ve süreç kontrolünü sağlar.

Teknolojiler: NestJS, TypeORM, PostgreSQL

Sorumluluklar:

- Dashboard metrikleri
- Başvuru süreçlerinin izlenmesi
- Nihai karar ve raporlama

Ana Rotalar:

- GET /admin/dashboard
- POST /admin/final-approve
- GET /admin/reports/accepted
- GET /admin/reports/rejected

GENEL SİSTEM YAPISI

Monolitik mimari kapsamında tüm modüller tek bir uygulama içerisinde çalışmakta olup, veri erişimi ortak bir veritabanı üzerinden sağlanmaktadır. Modüller arası iletişim doğrudan fonksiyon çağrıları ile gerçekleştirilmekte, bu sayede mikroservis mimarisine kıyasla daha düşük gecikme süresi elde edilmektedir.

Bu yapı, özellikle geliştirme sürecinde hızlı ilerleme, kolay hata ayıklama ve düşük operasyonel maliyet avantajı sağlamaktadır. Aynı zamanda sistemin modüler olarak tasarlanmış olması, ilerleyen aşamalarda mikroservis mimariye geçiş için uygun bir temel oluşturmaktadır.

Lisansüstü Başvuru Sistemi projesinin mimari tasarım aşaması, ölçeklenebilirlik, bakım kolaylığı, bağımsız geliştirme ve dağıtım ihtiyaçlarını karşılamak amacıyla mikroservis mimarisi üzerine kurulmuştur. Bu yaklaşım, sistemin her bileşenini ayrı bir servis olarak yapılandırarak, servisler

arası iletişimi senkron (REST API) ve asenkron (RabbitMQ mesaj kuyruğu) yöntemlerle sağlar. Tüm dış istekler, bir API Gateway katmanı üzerinden yönlendirilir. Proje, 17.12.2025 tarihli dokümanda detaylandırıldığı üzere, NestJS backend, React frontend ve PostgreSQL veritabanı gibi teknolojileri entegre eder. Aşağıda, mikroservislerin sorumlulukları, teknolojileri, ana rotaları ve genel sistem akışı detaylı olarak açıklanmıştır. Bu yazı, projenin mikroservis yapısını temel alarak hazırlanmış olup, geliştirme sürecine rehberlik eder. [3], [4],[12]

9.1. MİKROSERVİSLER VE SORUMLULUKLARI

Proje, toplam 8 ana mikroservisten oluşur. Her servis, belirli bir işlevselliğe odaklanır ve bağımsız olarak ölçeklenebilir. Servisler, NestJS framework'ü ile geliştirilmiş olup, TypeORM aracılığıyla PostgreSQL veritabanına bağlanır. Asenkron işlemler için RabbitMQ, oturum yönetimi için Redis kullanılır. [12], [13], [8], [9]

- **Auth Service (Kimlik Doğrulama Servisi):** Kullanıcı kimlik doğrulaması ve yetkilendirme işlemlerini yönetir.
 - **Teknolojiler:** NestJS (Node.js), TypeORM, PostgreSQL, Redis (token blacklist ve oturum yönetimi için).
 - **Sorumluluklar:** Kullanıcı kayıt, giriş, JWT token oluşturma/yenileme, rol bazlı yetkilendirme, e-posta doğrulama, şifre sıfırlama, Süper Yönetici yönetimi.[6], [7]
 - **Ana Rotalar:**
 - POST /auth/register (kayıt).
 - POST /auth/login (giriş).
 - POST /auth/refresh-token (token yenileme).
 - POST /auth/verify-email (e-posta doğrulama).
 - POST /auth/forgot-password (şifre sıfırlama).
 - POST /auth/2fa-verify (opsiyonel iki faktörlü doğrulama).

Şifre Politikası Önerisi: Parolalar en az 8 karakter uzunluğunda olacak şekilde belirlenir. Kullanıcıların tahmin edilmesi zor, ancak hatırlanabilir parolalar oluşturması teşvik edilir. Gerekli durumlarda büyük harf, küçük harf ve rakam

kullanımı desteklenir. Yaygın, kolay tahmin edilebilir veya daha önce ele geçirilmiş parolaların kullanımı sistem tarafından engellenir.

Kullanıcı parolalarının belirli aralıklarla güncellenmesi sağlanır. Bu kapsamda parolalar 4 veya 6 ayda bir değiştirilir. Bu uygulama, uzun süre aynı parolanın kullanılmasından kaynaklanabilecek güvenlik risklerini azaltmayı amaçlar. Yetkili kullanıcı hesapları için ek güvenlik önlemleri uygulanabilir. Daha önce kullanılan parolaların tekrar kullanılması sınırlandırılır. Belirli sayıda hatalı giriş denemesinden sonra hesap geçici olarak kilitlenir. Uzun süre kullanılmayan kullanıcı hesapları pasif duruma alınır. Kullanıcı parolaları sistem içerisinde düz metin olarak saklanmaz ve güvenli şekilde korunur.

- **User Service (Kullanıcı Yönetimi Servisi):** Kullanıcı profilleri ve rollerini yönetir.
 - **Teknolojiler:** NestJS, TypeORM, PostgreSQL.
 - **Sorumluluklar:** Aday profili yönetimi, komisyon üyesi bilgileri, rol atama (Süper Yönetici hariç).
 - **Ana Rotalar:**
 - GET /users/profile (profil görüntüleme).
 - PUT /users/profile (profil güncelleme).
 - GET /users (admin only – kullanıcı listeleme).
 - PUT /users/:id/role (admin only – rol düzenleme).
- **Program Service (Program Yönetimi Servisi):** Lisansüstü programları ve kontenjanları tanımlar.
 - **Teknolojiler:** NestJS, TypeORM, PostgreSQL.
 - **Sorumluluklar:** Program tanımlama, kontenjan yönetimi, başvuru takvimi ve kriter belirleme.
 - **Ana Rotalar:**
 - GET /programs (program listeleme).

- POST /programs (admin only – program ekleme).
 - PUT /programs/:id (admin only – program güncelleme).
 - DELETE /programs/:id (admin only – program silme).
- **Application Service (Başvuru Servisi):** Başvuru süreçlerini yönetir.
 - **Teknolojiler:** NestJS, TypeORM, PostgreSQL, RabbitMQ (event yayınlama).
 - **Sorumluluklar:** Başvuru oluşturma, güncelleme, durum takibi, form validasyonu.
 - **Ana Rotalar:**
 - POST /applications (başvuru oluşturma).
 - GET /applications/my (adayın kendi başvuruları).
 - GET /applications/:id (başvuru detay).
 - PUT /applications/:id (başvuru güncelleme).
 - GET /applications/assigned (komisyon üyesi – atanan başvurular).
- **Document Service (Belge Yönetimi Servisi):** Belgelerin yüklenmesi ve yönetimini sağlar.
 - **Teknolojiler:** NestJS, Multer (yerel depolama veya S3/MinIO), RabbitMQ (opsiyonel OCR işleme).
 - **Sorumluluklar:** Belge yükleme, format/boyut kontrolü, önizleme, silme, hash doğrulama.
 - **Ana Rotalar:**
 - POST /documents/upload (multipart/form-data – belge yükleme).
 - GET /documents/:id/view (stream – belge görüntüleme).
 - DELETE /documents/:id (belge silme).
 - GET /applications/:id/documents (başvuruya ait belgeler).

- **Evaluation Service (Değerlendirme Servisi):** Komisyon değerlendirmelerini yönetir.
 - **Teknolojiler:** NestJS, TypeORM, PostgreSQL, RabbitMQ.
 - **Sorumluluklar:** Komisyon atama, puanlama, otomatik ağırlıklı ortalama hesaplama, karar verme.
 - **Ana Rotalar:**
 - GET /evaluations/my (komisyon üyesi – kendi değerlendirmeleri).
 - POST /evaluations (değerlendirme oluşturma).
 - PUT /evaluations/:id (değerlendirme güncelleme).
- **Notification Service (Bildirim Servisi):** Otomatik bildirimleri gönderir.
 - **Teknolojiler:** NestJS, RabbitMQ (consumer), Nodemailer veya SendGrid.
 - **Sorumluluklar:** E-posta bildirimleri, kuyruk yönetimi, bildirim geçmişi tutma.
 - **Ana Rotalar:**
 - GET /notifications (kullanıcıya özel bildirimler).
 - PUT /notifications/:id/read (bildirim okundu olarak işaretleme).
 - **Not:** Servis, RabbitMQ'dan gelen mesajları asenkron işler.
- **Admin Service (Yönetim Servisi):** Sistem yönetimini sağlar.
 - **Teknolojiler:** NestJS, TypeORM, PostgreSQL.
 - **Sorumluluklar:** Dashboard metrikleri, süreç izleme, nihai onay, rapor üretimi.
 - **Ana Rotalar:**
 - GET /admin/dashboard (dashboard metrikleri).
 - POST /admin/final-approve (nihai onay).

- GET /admin/reports/accepted (kabul raporları).
- GET /admin/reports/rejected (ret raporları).

9.2. FRONTEND UYGULAMASI

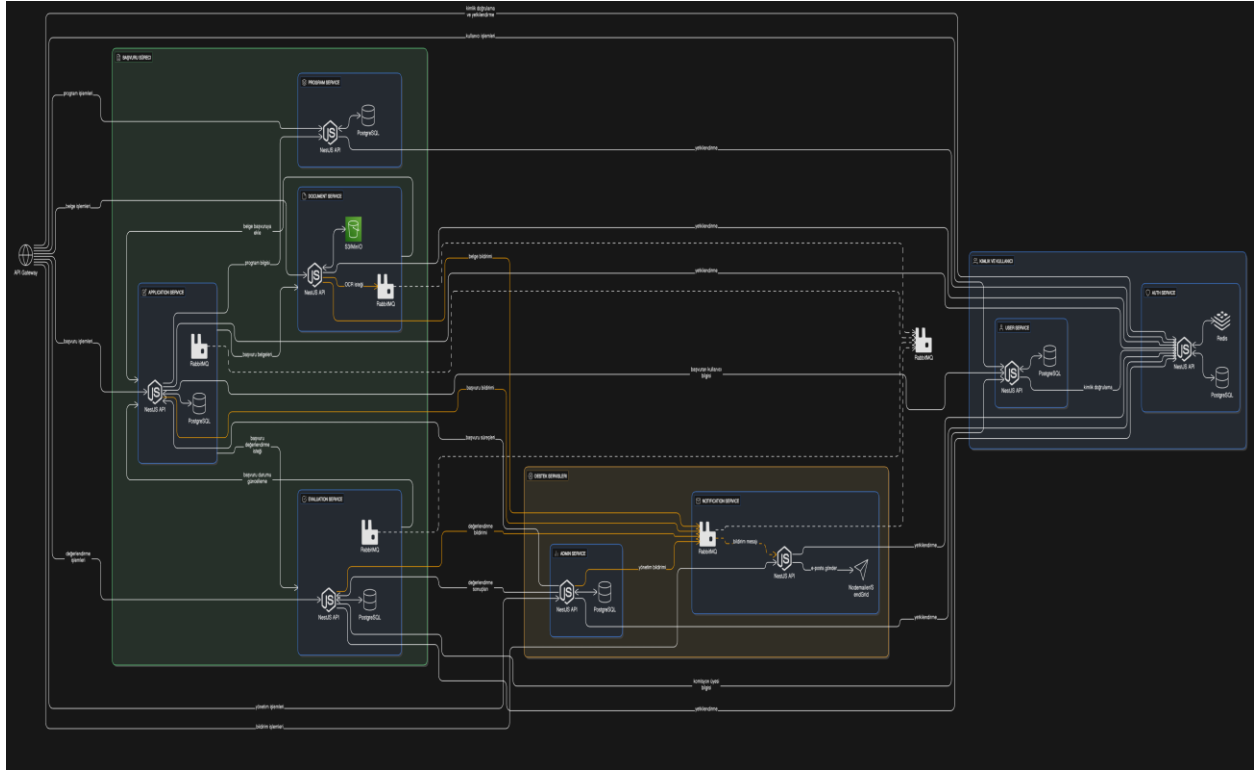
Frontend, mikroservislerle API Gateway üzerinden iletişim kurar.

- **Teknolojiler:** Next.js (React), TailwindCSS, Axios veya React Query.
- **Örnek Sayfa Rotaları:**
 - /login (giriş).
 - /register (kayıt).
 - /dashboard (rol bazlı yönlendirme).
 - /apply/[programId] (başvuru yapma).
 - /my-applications (başvurularım).
 - /commission/evaluations (değerlendirmeler).
 - /admin/programs (program yönetimi).
 - /admin/dashboard (yönetim dashboardu).

9.3. SERVİS ŞEMASI

Sistem şeması, mikroservislerin birbirleriyle etkileşimini gösterir: Auth Service tüm servisler için kimlik doğrulamayı sağlar, Application Service olayları RabbitMQ ile Notification Service'e yayınlar, Document Service dosyaları depolar ve Evaluation Service puanlamaları Admin Service'e iletir. API Gateway, yük dengeleme ve güvenlik katmanı ekler. Bu yapı, yüksek yük dönemlerinde (başvuru haftaları) performansı korur.

Bu mikroservis tasarımı, projenin SDLC sürecinde (tasarım ve geliştirme aşamaları) uygulanmış olup, NestJS'in modüler yapısı sayesinde genişletilebilirlik sağlar. Gelecekteki iyileştirmelerde (örn. mobil entegrasyon veya ek servisler), bu şema temel alınarak güncellenebilir.



10. KULLANILAN TEKNOLOJİLER VE ARAÇLAR

Lisansüstü Başvuru Sistemi projesi, modern full-stack JavaScript/TypeScript yaklaşımıyla geliştirilmiştir. Sistem, **NestJS** tabanlı ölçeklenebilir bir backend mimarisi, **React** tabanlı dinamik bir frontend ve ilişkisel veritabanı entegrasyonu üzerine kurulmuştur. 2026 yılı itibarıyla bu

teknoloji yığını, kurumsal web uygulamaları için en popüler ve güvenilir seçeneklerden biri olarak kabul edilmektedir. Aşağıda, proje kapsamında kullanılan teknolojiler ve araçlar detaylı olarak listelenmiştir.

10.1. FRONTEND TEKNOLOJİLERİ

Frontend katmanı, kullanıcı dostu, responsive ve yüksek performanslı arayüzler sağlamak amacıyla modern araçlarla yapılandırılmıştır.

- **React.js:** Ana frontend kütüphanesi olarak component tabanlı mimariyi destekler. Yeniden kullanılabilir bileşenler sayesinde bakım ve geliştirme süreci hızlandırılır.[10]
- **Tailwind CSS:** Utility-first CSS framework'ü ile hızlı, özelleştirilebilir ve responsive tasarım oluşturulur. Modern arayüzler için ideal bir seçenektir.
- **React Query (TanStack Query):** Veri çekme (fetching), önbellekleme (caching), state yönetimi ve senkronizasyon işlemlerini kolaylaştırır.[10]
- **Axios:** HTTP istekleri için güvenilir ve yaygın kullanılan client kütüphanesidir.
- **React Hook Form:** Dinamik form yönetimi, validasyon ve belge yükleme gibi işlemleri verimli hale getirir.[10]
- **React Router:** Sayfa yönlendirme, rol bazlı rota koruması ve navigasyon için kullanılır.[10]

10.2. BACKEND TEKNOLOJİLERİ

Backend, modüler, tip güvenli ve kurumsal standartlara uygun bir yapıya sahiptir.

- **NestJS (Node.js):** Projenin ana backend framework'üdür. TypeScript tabanlı, modüler mimari, dependency injection ve decorator yapısı sayesinde temiz ve ölçeklenebilir kod yazımı sağlar. Mikroservis mimarisi için mükemmel uyum gösterir.[9]
- **TypeORM:** PostgreSQL veritabanı ile entegrasyon, entity tanımlama, migration yönetimi ve ilişkisel sorgular için kullanılır.

- **JWT (JSON Web Token):** Güvenli kimlik doğrulama ve yetkilendirme mekanizması olarak tercih edilmiştir.[6], [7]
- **bcrypt / argon2:** Kullanıcı şifrelerinin güvenli hash'lenmesi için kullanılır (argon2 daha modern ve önerilen seçenektir).
- **Multer:** Belge yükleme işlemleri (multipart/form-data) için middleware sağlar.
- **Nodemailer:** Otomatik e-posta bildirimleri göndermek amacıyla kullanılır (alternatif olarak SendGrid entegrasyonu da desteklenir).

10.3. VERİ TABANI VE ALTYAPI

Veri tutarlılığı, performans ve asenkron işlemler için seçilen teknolojiler:

- **PostgreSQL:** İlişkisel veritabanı olarak ana depolama çözümüdür. Veri bütünlüğü, ACID uyumluluğu ve karmaşık sorgular için idealdir.[8]
- **Redis:** Oturum yönetimi, token blacklist ve caching işlemleri için kullanılır.[13]
- **RabbitMQ:** Asenkron iletişim ve mesaj kuyruğu yönetimi için tercih edilmiştir. Bildirimler, olay tabanlı işlemler (event-driven) ve yük dengeleme senaryolarında kritik rol oynar.[12]

10.4. Diğer Araçlar ve Altyapı

Projenin geliştirme, test, dağıtım ve bakım süreçlerini destekleyen ek araçlar:

- **Docker & Docker Compose:** Uygulama containerization ve ortam tutarlılığı için kullanılır. Geliştirme, test ve production ortamlarında aynı yapılandırmayı garanti eder.
- **TypeScript:** Tüm kod tabanında tip güvenliği sağlamak amacıyla zorunlu olarak uygulanmıştır.
- **Jest:** Unit ve integration testleri için test framework'üdür.

Bu teknoloji yığını, projenin **gereksinim analizi, tasarım, geliştirme** ve **ölçeklenebilirlik** ihtiyaçlarını karşılayacak şekilde seçilmiştir. Özellikle NestJS + TypeORM + PostgreSQL + RabbitMQ kombinasyonu, 2026 yılı modern mikroservis tabanlı web uygulamalarında yaygın olarak kullanılan ve yüksek performans sağlayan bir staktır. Frontend'de React + TailwindCSS ise hızlı geliştirme ve kullanıcı odaklı arayüzler için en güncel tercihler arasındadır.

Gelecekteki genişletmeler (mobil uygulama entegrasyonu, AI destekli özellikler vb.) için bu yığın kolayca genişletilebilir niteliktedir. Proje, SDLC sürecinin tasarım ve geliştirme aşamalarında bu teknolojilerle hayata geçirilmiştir.

11.GELİŞTİRME SÜRECİ

11.1 Varsayılan Sunucu Adresi

- Adres: <http://localhost:3000>
- Port, PORT ortam değişkeni ile değiştirilebilir.

11.2. Projenin Kısa Tanımı

NestJS tabanlı REST API uygulamasıdır.

PostgreSQL + TypeORM ile veri katmanı, JWT + Passport ile kimlik doğrulama, class-validator ile istek doğrulama, Multer ile dosya yükleme, nodemailer ile e-posta gönderme ve yüklenen dosyalar için statik HTTP sunumu içermektedir.

Amaç: Enstitü ve program tanımları, kullanıcı rolleri, aday başvuruları, belge yükleme yolları ve komisyon karar süreçlerini uçtan uca desteklemek.

11.3. Teknoloji Özet Tablosu

Kategori	Paket / Araç	Açıklama
Çatı	NestJS 11	Modüler HTTP API, dependency injection, guard/interceptor
Dil	TypeScript 5.x	Derleme zamanı tip güvenliği
HTTP Sunucusu	@nestjs/platform-express	Express üzerinde Nest uygulaması
ORM	@nestjs/typeorm, typeorm	Entity, repository, ilişkiler, PostgreSQL bağlantısı
Veritabanı	pg	PostgreSQL sürücüsü

Kimlik Doğrulama	JWT, Passport, passport-jwt	Token üretimi ve doğrulama
Şifreleme	bcrypt	Hash üretimi ve karşılaştırma
DTO Doğrulama	class-validator, class-transformer	Pipe ile gövde doğrulama ve yanıt dönüşümü
Dosya Yükleme	Multer, @types/multer	Disk üzerine çoklu alan yükleme
E-posta	nodemailer	Doğrulama kodu vb. gönderme
Ortam Değişkenleri	dotenv	process.env yükleme
Statik Dosya Sunumu	@nestjs/serve-static	Yüklenen dosyalara URL ile erişim
Geliştirme	Jest, ESLint, Prettier, Supertest	Test ve kod kalitesi

11.4. Ortam Değişkenleri Tablosu

Değişken	Zorunluluk	Açıklama
PORT	İsteğe bağlı	Dinlenecek port; belirtilmezse 3000 kullanılır.
DATABASE_URL	İsteğe bağlı	Belirtilmezse kod içindeki varsayılan PostgreSQL bağlantısı kullanılır.
JWT_SECRET	Üretimde zorunlu	Token imzalama anahtarı; belirtilmezse zayıf varsayılan kullanılır.

Not: Veritabanı şema adı belek_graduate_admission olarak varsayılandır. synchronize: false (DDL otomatik oluşturulmaz). Bağlantı URL'sinde sslmode=require varsa SSL etkinleştirilir.

11.5. Genel Uygulama Ayarları (main.ts)

Özellik	Açıklama
ValidationPipe (global)	transform: true, whitelist: true, forbidNonWhitelisted: true
CORS	origin: true, credentials: true İzin verilen metodlar: GET, HEAD, PUT, PATCH, POST, DELETE

11.6. Veri Modeli İlişki Özeti

Varlık	Önemli Alanlar / İlişkiler
Role	name, description; User listesi (OneToMany)
User	email, passwordHash, Role (ManyToOne)
Institute	name; oluşturan User
Program	name, quota, dates, Institute (ManyToOne), oluşturan User
Application	Başvuru tarihi, durum, GPA, dosya yolları; User (ManyToOne), Program (ManyToOne)
Decision	Karar metni/tarihi; Application (OneToOne), komisyon üyesi User

11.7. Modül Bazında Açıklama ve API Tablosu

Not: Tüm yollar uygulama köküne (/) göreler. Örnek: /auth/login → <http://localhost:3000/auth/login>.

JWT: Authorization: Bearer <token> başlığı gereklidir.

11.7.1. AppModule + AppController

- **AppModule:** TypeORM kök yapılandırması, tüm feature modüllerin importu, ServeStaticModule (yüklenen dosyalar için).
- **AppController:** Kök GET / → basit karşılama mesajı.

Statik Dosya Sunumu

Özellik	Değer
HTTP kök yolu	/documents
Dosya sistemi klasörü	src/documents/Uploads
Dizin listesi	index: false

11.7.2. AuthModule — Controller: auth

Yol	Metot	JWT	Kısa Açıklama
/auth/login	POST	Hayır	E-posta ve şifre ile giriş, access_token döner
/auth/register	POST	Hayır	Yeni kullanıcı kaydı (roleId servisi tarafından yönetilir)
/auth/change-password	POST	Evet	Mevcut ve yeni şifre değiştirme
/auth/send-verification-code	POST	Hayır	E-postaya doğrulama kodu gönderimi
/auth/verify-email	POST	Hayır	Kod ile e-posta doğrulama

11.7.3. RolesModule — Controller: roles

Yol	Metot	JWT	Rol / Not
/roles	POST	Hayır	(Üretimde admin kısıtlaması önerilir)
/roles	GET	Hayır	Tüm roller
/roles/:id	GET	Hayır	Tek rol
/roles/:id	PUT	Evet	admin

/roles/:id	DELETE	Evet	admin
------------	--------	------	-------

11.7.4. UsersModule — Controller: users

Yol	Metot	JWT	Rol / Kural
/users	POST	Evet	admin
/users	GET	Hayır	Tüm kullanıcılar (erişim politikası gözden geçirilmeli)
/users/me	GET	Evet	Oturumdaki kullanıcı (sınırlı alanlar)
/users/:id	GET	Hayır	Tek kullanıcı
/users/:id	PUT	Evet	admin veya kendi kaydı (role'de sadece admin)
/users/:id/reset-password	POST	Evet	admin veya kendi kaydı
/users/:id	DELETE	Evet	admin veya kendi kaydı

11.7.5. InstituteModule — Controller: institutes (sınıf düzeyinde JWT)

Yol	Metot	Admin Zorunlu	Açıklama
/institutes	POST	Evet	Enstitü oluştur

/institutes	GET	Evet	Listele
/institutes/:id	GET	Evet	Tekil
/institutes/:id	PUT	Evet	Güncelle
/institutes/:id	DELETE	Evet	Sil

11.7.6. ProgramsModule — Controller: programs (sınıf düzeyinde JWT)

Yol	Metot	Admin (işlem bazında)	Açıklama
/programs	POST	Evet	Program oluştur
/programs	GET	Hayır (JWT yeterli)	Listele
/programs/:id	GET	Hayır (JWT yeterli)	Tekil
/programs/:id	PUT	Evet	Güncelle
/programs/:id	DELETE	Evet	Sil

11.7.7. ApplicationsModule — Controller: applications (sınıf düzeyinde JWT)

Yol	Metot	Erişim Özeti
/applications/me	GET	Giriş yapan kullanıcının kendi başvuruları
/applications	POST	Normal kullanıcı: userId oturumdan; admin: userId gövdeden veya varsayılan
/applications	GET	Admin veya Komisyon Üyesi: tüm liste; diğerleri: filtreli liste
/applications/:id	GET	Sahip veya admin veya Komisyon Üyesi
/applications/:id	PATCH	Sahip veya admin
/applications/:id	DELETE	Sadece admin

11.7.8. DocumentsModule — Controller: upload

Yol	Metot	JWT	Açıklama
/upload/documents	POST	Guard yok	Form alanları: diploma, transcript, ydt (her biri max 1 dosya)

Üretim Önerisi: JWT koruması, dosya boyutu ve MIME türü kısıtları eklenmelidir.

11.7.9. DesicionModule — Controller: decisions

Yol	Metot	Rol / Erişim
/desicions	POST	Komisyon Üyesi
/desicions/:id	GET	JWT (rol detayı kodda kontrol edilmeli)
/desicions/:id	PUT	Komisyon Üyesi
/desicions/:id	DELETE	admin

11.8. Modül Dışı Genel Yapılar

- **main.ts:** Uygulamanın başlangıç noktasıdır. Global ValidationPipe ve CORS ayarları burada yapılır.
- **JwtAuthGuard & JwtStrategy:** Tüm korumalı endpoint'lerde kimlik doğrulama ve yetkilendirme sağlar. Token payload'ı (id, email, role) req.user nesnesine aktarılır.
- **Entity ve DTO Katmanları:** Her modülün kendi entity ve DTO dosyaları bulunur. Bu dosyalar veritabanı şeması ile HTTP istek/yanıt yapısını tanımlar.
- **AppModule Yapılandırması:** TypeORM veritabanı bağlantısı ve ServeStaticModule (dosya sunumu) burada tanımlanır.

11.9. Modüller Arası İlişkiler ve Bağımlılıklar

User ve Role Merkezi Rolü User varlığı sistemin merkezindedir. Auth, Users, Institute, Program, Application ve Decision modülleriyle ilişkilidir. Role varlığı ise yetki kontrollerinin temelini oluşturur.

Modüller Arası İlişki Özeti:

- **AuthModule:** UsersModule ve e-posta servisi ile çalışır. Token üretimi ve doğrulama burada yapılır.
- **UsersModule:** RolesModule ile entegredir. Kullanıcı oluşturma ve rol atama işlemleri paylaşılır.
- **InstituteModule & ProgramsModule:** Akademik yapı tanımlar. Oluşturan kullanıcı (createdBy) ilişkisi vardır.
- **DocumentsModule:** Belgeleri diske kaydeder. ApplicationsModule bu dosya yollarını doğrular.
- **ApplicationsModule:** Kullanıcı, program ve belgeleri birleştirir. Decision ile OneToOne ilişki içerir.
- **DesicionModule:** Başvurulara komisyon kararı ekler.

Basit İlişki Şeması (Metin):

- Kullanıcı Yönetimi: **Auth ↔ Users ↔ Roles**
- Akademik Yapı: **Institute → Program** (oluşturan User ile)
- Başvuru Süreci: **User + Program + Belgeler → Application**
- Komisyon Süreci: **Application ↔ Decision** (komisyon üyesi User ile)
- Dosya Erişimi: **DocumentsModule ↔ ServeStaticModule (/documents)**

11.10. Her Modülün Sorumluluğu

- **AppModule:** Tüm modülleri birleştirir, veritabanı ve statik dosya ayarlarını yönetir.
- **AuthModule:** Kimlik doğrulama, token yönetimi ve e-posta akışlarını gerçekleştirir.
- **UsersModule:** Kullanıcı hesaplarının oluşturulması, güncellenmesi ve yönetimini sağlar.
- **RolesModule:** Sistem rollerini tanımlar ve yönetir.
- **InstituteModule:** Enstitü kayıtlarını yönetir.
- **ProgramsModule:** Lisansüstü programlarını tanımlar.
- **DocumentsModule:** Adayların belgelerini yükler ve saklar.
- **ApplicationsModule:** Başvuru işlemlerini uçtan uca yönetir.
- **DesicionModule:** Komisyon kararlarını kaydeder ve yönetir.

11.11. Tipik Süreç Akışı

1. Roller ve kullanıcılar tanımlanır (admin rolü ile).
2. Enstitü ve program kayıtları oluşturulur.
3. Aday üye olur / giriş yapar → JWT token alır.
4. Belgeler /upload/documents ile yüklenir; dönen path'ler başvuru kaydına eklenir.
5. /applications ile başvuru oluşturulur veya güncellenir.
6. Komisyon üyesi /desicions ile karar oluşturur / günceller.

KAYNAKÇA

- [1] R. S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill Education, 2014.
- [2] I. Sommerville, Software Engineering, 10th ed., Pearson Education, 2016.
- [3] M. Fowler, Microservices: A Definition of This New Architectural Term, 2014.
- [4] S. Newman, Building Microservices: Designing Fine-Grained Systems, O'Reilly Media, 2021.
- [5] Object Management Group, Unified Modeling Language (UML) Specification, 2017.
- [6] Internet Engineering Task Force, RFC 7519: JSON Web Token (JWT), 2015.
- [7] OWASP Foundation, Authentication Cheat Sheet, 2023.
- [8] PostgreSQL Global Development Group, PostgreSQL Documentation, 2024.
- [9] NestJS Team, NestJS Documentation, 2024.
- [10] React Team, React Documentation, 2024.
- [11] Tailwind Labs, Tailwind CSS Documentation, 2024.
- [12] RabbitMQ Team, RabbitMQ Documentation, 2024.
- [13] Redis Ltd., Redis Documentation, 2024.
- [14] Kişisel Verileri Koruma Kurumu, 6698 Sayılı KVKK, 2024.
- [15] European Union, General Data Protection Regulation (GDPR), 2018.

